

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
CENTRO UNIVERSITARIO CHIQUIMULILLA, SANTA ROSA
FACULTAD DE INGENIERIA EN SISTEMAS

CURSO: DESARROLLO WEB

CATEDRÁTICO: NG. CARMELO ESTUARDO MAYÉN MONTERROSO

PROYECTO FINAL

MANUAL TECNICO DE GESTION DE NOMINA



ESTUDIANTES: Guillermo José Gómez Aguilera

Teddy Leonardo Hernández Pérez

Wilson Eduardo Hernández López

Gelen Dayanna López Morales

INDICE

1. Introducción.....	5
1.1 Objetivo del Manual	5
1.2 Alcance	5
1.3 Público Objetivo	5
2. Descripción general del sistema.....	5
2.1 Propósito del sistema.....	5
2.2 Cumplimiento legal Guatemala 2025	6
2.3 Tecnologías utilizadas	6
3. Arquitectura del frontend	7
3.1 Estructura de carpetas y archivos	7
3.2 Componentes principales	8
3.3 Hooks personalizados	8
3.4 Integración con backend	9
4. Instalación y configuración.....	9
4.1 Prerrequisitos	9
4.2 Clonado del repositorio.....	9
4.3 Instalación de dependencias	10
4.4 Configuración del entorno (.env)	10
4.5 Ejecución del entorno de desarrollo	10
5. Scripts disponibles.....	10
5.1 npm run dev	11
5.2 npm run build	11
5.3 npm run preview	11
5.4 npm run lint.....	11
5.5 npm run format.....	11
6. Funcionalidades del sistema.....	11
6.1 Gestión de empleados	12
6.2 Gestión de departamentos	12
6.3 Nóminas ordinarias.....	12
6.4 Nóminas extraordinarias	12
6.5 Dashboard y reportes	12
6.6 Seguridad y roles.....	13

7. Validaciones legales y cálculos automáticos	13
7.1 Validación de salario mínimo	13
7.2 Cálculo de IGSS.....	13
7.3 Cálculo de ISR	13
7.4 Aguinaldo y bono 14	14
7.5 Validación de antigüedad	14
7.6 Verificadores y herramientas de prueba.....	14
8. Componentes y hooks destacados	14
8.1 Componentes clave	14
CumplimientoLegal.tsx.....	14
CalculadoraNomina.tsx	15
FormularioEmpleado.tsx	15
TablaEmpleados.tsx.....	15
Dashboard.tsx.....	15
8.2 Hooks personalizados	15
useValidacionCumplimiento.ts.....	15
useCalculoNominaVerificacion.ts	15
useEmpleado.ts	16
useDepartamento.ts.....	16
9. Testing y verificación.....	16
9.1 Validaciones en tiempo real.....	16
9.2 Simuladores internos	16
9.3 Herramientas de desarrollo	17
9.4 Pruebas manuales	17
10. Endpoints consumidos.....	17
10.1 Configuración base	17
10.2 Endpoints principales	17
10.3 Ejemplo de consumo.....	18
11. Buenas prácticas y recomendaciones	19
11.1 Organización del código	19
11.2 Validaciones estrictas.....	19
11.3 Manejo de datos eficiente	19
11.4 Estilos coherentes.....	19

11.5 Seguridad y control de acceso	20
11.6 Flujo de trabajo con Git	20
12. Contribución y mantenimiento	20
12.1 Cómo colaborar	20
12.2 Flujo de trabajo recomendado	20
12.3 Mantenimiento del sistema	21
15. Cierre	21

1. Introducción

1.1 Objetivo del Manual

Este manual técnico tiene como objetivo documentar de forma detallada y profesional la estructura, configuración, funcionalidades y componentes clave del frontend del sistema de nóminas desarrollado con React y TypeScript. Está orientado a desarrolladores, técnicos y personal de mantenimiento que requieran comprender, modificar o escalar el sistema.

1.2 Alcance

El manual cubre exclusivamente el módulo frontend del sistema de nóminas, el cual se comunica con una API REST desarrollada en ASP.NET Core. Se documentan las tecnologías utilizadas, la arquitectura del proyecto, los componentes clave, la configuración del entorno de desarrollo, y las funcionalidades implementadas para cumplir con la legislación laboral vigente en Guatemala en el año 2025.

1.3 Público Objetivo

Este documento está dirigido a:

- Desarrolladores frontend que deseen mantener o extender el sistema.
- Equipos de QA encargados de pruebas funcionales y de interfaz.
- Personal técnico de soporte o implementación.
- Docentes y evaluadores del proyecto universitario.

2. Descripción general del sistema

2.1 Propósito del sistema

El sistema de nóminas desarrollado en este proyecto tiene como propósito principal automatizar, validar y gestionar de forma integral todos los procesos relacionados con la administración de nóminas en instituciones guatemaltecas, cumpliendo rigurosamente con la legislación laboral vigente en el país para el año 2025. Este frontend actúa como la interfaz visual y funcional que permite a los usuarios interactuar con el sistema, realizar cálculos complejos, verificar cumplimiento legal, generar reportes y administrar empleados, departamentos y pagos extraordinarios.

El sistema no solo busca eficiencia operativa, sino también transparencia, trazabilidad y seguridad en el manejo de datos sensibles. Cada módulo ha sido diseñado para reflejar las

necesidades reales de empresas guatemaltecas, incluyendo el cálculo automático de IGSS, ISR, aguinaldo, bono 14, salario mínimo, y otros componentes legales que afectan directamente la nómina de los trabajadores.

2.2 Cumplimiento legal Guatemala 2025

Una de las características más destacadas del sistema es su enfoque en el cumplimiento legal. El frontend incorpora validadores y calculadoras que se actualizan conforme a las disposiciones del Ministerio de Trabajo y Previsión Social de Guatemala, incluyendo:

- Cálculo automático de deducciones de IGSS e ISR según tablas oficiales.
- Verificación del cumplimiento del salario mínimo vigente por sector económico.
- Validación de antigüedad para el cálculo proporcional de aguinaldo y bono 14.
- Generación de nóminas ordinarias, extraordinarias y especiales con respaldo legal.
- Alertas visuales y mensajes de error en caso de incumplimiento normativo.

Este enfoque garantiza que cada nómina generada desde el sistema esté respaldada por la ley, reduciendo riesgos legales y fortaleciendo la confianza institucional.

2.3 Tecnologías utilizadas

El frontend ha sido construido con un stack moderno, robusto y altamente escalable. Las tecnologías utilizadas incluyen:

- **React:** Biblioteca principal para la construcción de interfaces reactivas y modulares.
- **TypeScript:** Lenguaje tipado que permite mayor seguridad, claridad y mantenibilidad del código.
- **Vite:** Herramienta de construcción ultrarrápida que optimiza el desarrollo y el rendimiento.
- **Tailwind CSS:** Framework de estilos utilitarios que permite una personalización visual precisa y coherente.
- **React Hook Form + Zod:** Para validaciones de formularios con tipado estricto y retroalimentación inmediata.
- **React Query:** Para manejo eficiente de datos asíncronos, caché y sincronización con el backend.

- **Axios:** Cliente HTTP para consumir endpoints REST de forma segura y estructurada.
- **Canva y Figma (opcional):** Para diseño visual de componentes y documentación gráfica.

Este stack permite una experiencia de usuario fluida, validaciones en tiempo real, y una arquitectura limpia que facilita el mantenimiento y la escalabilidad del sistema.

3. Arquitectura del frontend

3.1 Estructura de carpetas y archivos

El proyecto está organizado siguiendo buenas prácticas de desarrollo frontend profesional. La estructura base incluye:

Frontend-Nominas/

└─ public/

└─ src/

| └─ assets/

| └─ components/

| └─ hooks/

| └─ pages/

| └─ services/

| └─ utils/

| └─ App.tsx

| └─ main.tsx

└─ .env

└─ package.json

└─ tsconfig.json

└─ vite.config.ts

- **assets/:** Contiene imágenes, íconos y recursos visuales utilizados en la interfaz.
- **components/:** Componentes reutilizables como formularios, tablas, botones, modales, etc.

- **hooks/**: Hooks personalizados para validaciones legales, cálculos de nómina y lógica de negocio.
- **pages/**: Vistas principales del sistema como Dashboard, Empleados, Departamentos, Nóminas.
- **services/**: Configuración de Axios y funciones para consumir endpoints REST.
- **utils/**: Funciones auxiliares para cálculos, formatos y validaciones.

3.2 Componentes principales

Entre los componentes más relevantes se encuentran:

- **CumplimientoLegal.tsx**: Verifica si los datos ingresados cumplen con la legislación guatemalteca.
- **CalculadoraNomina.tsx**: Realiza cálculos automáticos de deducciones, bonificaciones y salario neto.
- **TablaEmpleados.tsx**: Muestra una tabla interactiva con filtros, paginación y acciones CRUD.
- **FormularioEmpleado.tsx**: Permite registrar y editar empleados con validaciones estrictas.
- **Dashboard.tsx**: Panel principal con indicadores clave, gráficas y accesos rápidos.

Cada componente está diseñado para ser reutilizable, modular y fácil de mantener.

3.3 Hooks personalizados

El sistema incluye hooks altamente especializados como:

- **useValidacionCumplimiento.ts**: Evalúa si los datos ingresados cumplen con salario mínimo, antigüedad, y deducciones legales.
- **useCalculoNominaVerificacion.ts**: Calcula automáticamente el salario neto, deducciones, bonificaciones y verifica que todo esté conforme a ley.
- **useDepartamento.ts**: Maneja la lógica de creación, edición y eliminación de departamentos.
- **useEmpleado.ts**: Administra el estado y lógica de los empleados registrados.

Estos hooks encapsulan la lógica de negocio, separando responsabilidades y facilitando la escalabilidad del sistema.

3.4 Integración con backend

El frontend se comunica con una API REST desarrollada en ASP.NET Core. La integración se realiza mediante Axios, con configuración centralizada en services/api.ts. Se consumen endpoints para:

- Autenticación y autorización con JWT.
- CRUD de empleados, departamentos y nóminas.
- Validación de datos legales.
- Generación de reportes y exportación de nóminas.

La comunicación es segura, eficiente y estructurada, permitiendo una sincronización fluida entre frontend y backend.

4. Instalación y configuración

4.1 Prerrequisitos

Antes de instalar el proyecto, es necesario contar con el siguiente entorno técnico:

- **Node.js v18 o superior:** Para ejecutar scripts y gestionar dependencias.
- **npm o yarn:** Gestor de paquetes para instalar módulos.
- **Editor de código (VS Code recomendado):** Para visualizar y modificar el código fuente.
- **Extensiones recomendadas:**
 - ESLint
 - Prettier
 - Tailwind CSS IntelliSense
 - TypeScript Hero

Además, se recomienda tener conocimientos básicos de React, TypeScript y consumo de APIs REST para una comprensión completa del sistema.

4.2 Clonado del repositorio

Para obtener el código fuente del frontend, se debe clonar el repositorio desde GitHub:

```
git clone https://github.com/GuillermoGome2z/Frontend-Nominas.git
```

```
cd Frontend-Nominas
```

Este comando descargará todos los archivos del proyecto y ubicará al usuario dentro de la carpeta raíz.

4.3 Instalación de dependencias

Una vez clonado el repositorio, se deben instalar las dependencias necesarias para ejecutar el proyecto:

```
npm install
```

Este comando instalará todos los módulos definidos en el archivo package.json, incluyendo React, Tailwind, Axios, React Query, React Hook Form, Zod, entre otros.

4.4 Configuración del entorno (.env)

El proyecto utiliza variables de entorno para definir la URL base del backend y otros parámetros sensibles. Se debe crear un archivo .env en la raíz del proyecto con el siguiente contenido:

```
VITE_API_URL=http://localhost:5000/api
```

Este valor puede cambiar según el entorno de desarrollo o producción. Es fundamental que la URL coincida con la dirección del backend ASP.NET Core que se esté utilizando.

4.5 Ejecución del entorno de desarrollo

Para iniciar el servidor de desarrollo y visualizar el sistema en el navegador, se debe ejecutar:

```
npm run dev
```

Este comando iniciará Vite y abrirá el sistema en <http://localhost:5173> (puerto por defecto). Cualquier cambio realizado en el código se reflejará automáticamente gracias al hot reload de Vite.

5. Scripts disponibles

El archivo package.json incluye varios scripts que automatizan tareas comunes durante el desarrollo. A continuación se detallan los más importantes:

5.1 npm run dev

- **Propósito:** Inicia el servidor de desarrollo con Vite.
- **Uso:** Se ejecuta durante el desarrollo para visualizar cambios en tiempo real.
- **Resultado:** Abre el sistema en el navegador y recarga automáticamente al modificar archivos.

5.2 npm run build

- **Propósito:** Genera una versión optimizada del proyecto para producción.
- **Uso:** Se ejecuta antes de desplegar el sistema en servidores públicos.
- **Resultado:** Crea una carpeta dist/ con los archivos modificados y listos para producción.

5.3 npm run preview

- **Propósito:** Simula el entorno de producción localmente.
- **Uso:** Permite verificar cómo se comporta el sistema una vez desplegado.
- **Resultado:** Inicia un servidor local que sirve los archivos de dist/.

5.4 npm run lint

- **Propósito:** Ejecuta ESLint para verificar errores de estilo y sintaxis.
- **Uso:** Se recomienda ejecutar antes de cada commit.
- **Resultado:** Muestra advertencias y errores que deben corregirse para mantener buenas prácticas.

5.5 npm run format

- **Propósito:** Aplica Prettier para formatear el código automáticamente.
- **Uso:** Mejora la legibilidad y coherencia del código fuente.
- **Resultado:** Reescribe los archivos con formato limpio y estandarizado.

6. Funcionalidades del sistema

El frontend del sistema de nóminas ha sido diseñado para ofrecer una experiencia de usuario intuitiva, eficiente y legalmente robusta. Las funcionalidades están organizadas en

módulos que reflejan los procesos reales de gestión de personal y nómina en instituciones guatemaltecas. A continuación se detallan las principales funcionalidades:

6.1 Gestión de empleados

- Registro de nuevos empleados con campos obligatorios como nombre, DPI, fecha de ingreso, salario base, departamento y tipo de contrato.
- Edición de datos existentes con validaciones estrictas.
- Eliminación segura con confirmación visual.
- Visualización en tabla con filtros por nombre, departamento y estado.
- Validación automática de salario mínimo según sector económico.

6.2 Gestión de departamentos

- Creación de departamentos con nombre y descripción.
- Asociación de empleados a departamentos específicos.
- Edición y eliminación de departamentos con control de dependencias.
- Visualización en tabla con paginación y búsqueda.

6.3 Nóminas ordinarias

- Generación de nómina mensual con cálculo automático de IGSS, ISR y salario neto.
- Validación de cumplimiento legal antes de guardar.
- Visualización de nómina por empleado con desglose de deducciones y bonificaciones.
- Exportación de nómina en formato imprimible.

6.4 Nóminas extraordinarias

- Generación de aguinaldo y bono 14 con cálculo proporcional según antigüedad.
- Validación de fechas de ingreso para determinar elegibilidad.
- Visualización de nómina extraordinaria con desglose legal.
- Exportación y respaldo de nómina especial.

6.5 Dashboard y reportes

- Panel principal con indicadores clave: total de empleados, nóminas generadas, cumplimiento legal, alertas.

- Gráficas interactivas de distribución salarial, cumplimiento por departamento y evolución de nóminas.
- Accesos rápidos a módulos críticos.
- Reportes descargables en PDF o impresión directa.

6.6 Seguridad y roles

- Autenticación mediante JWT con validación en frontend.
- Roles diferenciados: técnico (operativo) y coordinador (administrativo).
- Control de acceso a funcionalidades según rol.
- Redirección automática en caso de acceso no autorizado.

7. Validaciones legales y cálculos automáticos

Una de las fortalezas más destacadas del sistema es su capacidad para realizar validaciones legales y cálculos automáticos conforme a la normativa guatemalteca vigente. Estas validaciones están integradas en tiempo real y se ejecutan tanto en formularios como en procesos de nómina.

7.1 Validación de salario mínimo

- Verifica que el salario base ingresado cumpla con el mínimo legal según sector económico.
- Muestra alertas visuales si el salario es inferior al permitido.
- Bloquea el registro de empleados que no cumplen con esta condición.

7.2 Cálculo de IGSS

- Aplica el porcentaje oficial de deducción sobre el salario base.
- Calcula el aporte patronal y el aporte del empleado.
- Muestra el resultado en tiempo real en la nómina.

7.3 Cálculo de ISR

- Aplica la tabla de ISR vigente según el salario mensual.
- Calcula el impuesto a pagar y lo descuenta del salario neto.
- Muestra desglose en la nómina generada.

7.4 Aguinaldo y bono 14

- Calcula proporcionalmente según fecha de ingreso y salario base.
- Verifica antigüedad mínima para aplicar el beneficio.
- Muestra el monto correspondiente en la nómina extraordinaria.

7.5 Validación de antigüedad

- Calcula automáticamente los meses trabajados desde la fecha de ingreso.
- Determina elegibilidad para beneficios especiales.
- Muestra resultados en formularios y nóminas.

7.6 Verificadores y herramientas de prueba

- Hooks personalizados (useValidacionCumplimiento, useCalculoNominaVerificacion) que encapsulan toda la lógica legal.
- Formularios con validaciones en tiempo real usando Zod y React Hook Form.
- Mensajes de error claros y específicos para cada regla legal.
- Simuladores internos que permiten probar escenarios antes de guardar.

8. Componentes y hooks destacados

El sistema cuenta con una serie de componentes y hooks personalizados que encapsulan la lógica de negocio, validaciones legales y cálculos automáticos. Estos elementos han sido diseñados con enfoque modular, reutilizable y profesional, permitiendo una arquitectura limpia y escalable.

8.1 Componentes clave

CumplimientoLegal.tsx

- Verifica en tiempo real si los datos ingresados cumplen con la legislación guatemalteca.
- Evalúa salario mínimo, antigüedad, deducciones legales y beneficios extraordinarios.
- Muestra alertas visuales y mensajes de error específicos.
- Se integra con formularios de empleados y nóminas.

CalculadoraNomina.tsx

- Realiza cálculos automáticos de IGSS, ISR, salario neto, aguinaldo y bono 14.
- Utiliza funciones auxiliares y hooks para mantener la lógica separada.
- Muestra resultados en tiempo real conforme se ingresan los datos.
- Permite simulaciones antes de guardar la nómina.

FormularioEmpleado.tsx

- Formulario completo para registrar o editar empleados.
- Validaciones estrictas con Zod y React Hook Form.
- Campos obligatorios: nombre, DPI, salario, fecha de ingreso, departamento.
- Retroalimentación visual inmediata en caso de errores.

TablaEmpleados.tsx

- Tabla interactiva con paginación, filtros y acciones CRUD.
- Permite búsqueda por nombre, departamento y estado.
- Botones de acción para editar, eliminar o ver detalles.
- Integración con React Query para manejo eficiente de datos.

Dashboard.tsx

- Panel principal con indicadores clave del sistema.
- Gráficas interactivas y tarjetas informativas.
- Accesos rápidos a módulos críticos.
- Visualización de cumplimiento legal y alertas.

8.2 Hooks personalizados

useValidacionCumplimiento.ts

- Evalúa si los datos ingresados cumplen con las leyes laborales guatemaltecas.
- Verifica salario mínimo, antigüedad, deducciones y beneficios.
- Devuelve mensajes de error específicos y booleanos de validación.
- Se utiliza en formularios y componentes de nómina.

useCalculoNominaVerificacion.ts

- Calcula automáticamente salario neto, IGSS, ISR, aguinaldo y bono 14.

- Verifica que los cálculos estén dentro de los parámetros legales.
- Permite simulaciones previas a la generación de nómina.
- Se integra con CalculadoraNomina.tsx y FormularioEmpleado.tsx.

useEmpleado.ts

- Maneja el estado y lógica de los empleados registrados.
- Funciones para crear, editar, eliminar y listar empleados.
- Integración con React Query y Axios para consumo de API.
- Encapsula toda la lógica relacionada al módulo de empleados.

useDepartamento.ts

- Administra la lógica de creación, edición y eliminación de departamentos.
- Permite listar departamentos y asociarlos a empleados.
- Encapsula la lógica del módulo de departamentos.
- Se integra con formularios y tablas.

9. Testing y verificación

El sistema incluye mecanismos de verificación y pruebas que garantizan la calidad, estabilidad y cumplimiento legal del frontend. Aunque el repositorio no incluye pruebas automatizadas explícitas, se han implementado herramientas y estrategias que permiten validar el comportamiento del sistema en tiempo real.

9.1 Validaciones en tiempo real

- Formularios con validaciones estrictas usando Zod y React Hook Form.
- Mensajes de error específicos para cada regla legal.
- Retroalimentación visual inmediata en campos obligatorios.
- Bloqueo de acciones si los datos no cumplen con la ley.

9.2 Simuladores internos

- Componentes como CalculadoraNomina.tsx permiten simular escenarios antes de guardar.
- Verificadores legales que evalúan antigüedad, salario mínimo y beneficios.
- Hooks que devuelven resultados booleanos y mensajes explicativos.

9.3 Herramientas de desarrollo

- ESLint y Prettier para mantener estilo y coherencia del código.
- React Developer Tools para inspección de componentes y estado.
- Tailwind CSS IntelliSense para visualización de estilos.
- Axios Interceptors para monitorear llamadas a la API.

9.4 Pruebas manuales

- Validación de cada módulo mediante pruebas funcionales en navegador.
- Verificación de flujos completos: registro de empleado → generación de nómina → validación legal.
- Pruebas de bordes: salarios mínimos, fechas límite, antigüedad exacta.
- Simulación de errores y validación de mensajes de retroalimentación.

10. Endpoints consumidos

El frontend se comunica con una API REST desarrollada en ASP.NET Core, consumiendo múltiples endpoints que permiten realizar operaciones CRUD, validaciones legales y generación de nóminas. La integración se realiza mediante Axios, con configuración centralizada en el archivo `services/api.ts`.

10.1 Configuración base

```
const api = axios.create({  
  
  baseURL: import.meta.env.VITE_API_URL,  
  
  headers: {  
  
    'Content-Type': 'application/json',  
  
  },  
  
});
```

Esta configuración permite que todas las llamadas se dirijan a la URL definida en el archivo `.env`, facilitando la adaptación entre entornos de desarrollo y producción.

10.2 Endpoints principales

A continuación, se detallan los endpoints consumidos por el frontend:

Módulo	Método HTTP	Ruta	Descripción
Autenticación	POST	/auth/login	Inicia sesión y devuelve token JWT
Empleados	GET	/empleados	Obtiene listado completo de empleados
Empleados	POST	/empleados	Crea un nuevo empleado
Empleados	PUT	/empleados/{id}	Actualiza datos de un empleado
Empleados	DELETE	/empleados/{id}	Elimina un empleado
Departamentos	GET	/departamentos	Obtiene listado de departamentos
Departamentos	POST	/departamentos	Crea un nuevo departamento
Departamentos	PUT	/departamentos/{id}	Edita un departamento existente
Departamentos	DELETE	/departamentos/{id}	Elimina un departamento
Nóminas	GET	/nominas	Obtiene listado de nóminas generadas
Nóminas	POST	/nominas	Genera una nueva nómina ordinaria o extraordinaria
Nóminas	GET	/nominas/{id}	Obtiene detalles de una nómina específica
Validaciones	POST	/validaciones/cumplimiento	Verifica cumplimiento legal de datos ingresados
Cálculos	POST	/cálculos/nomina	Calcula deducciones, bonificaciones y salario neto

10.3 Ejemplo de consumo

```
const obtenerEmpleados = async () => {
  const response = await api.get('/empleados');
  return response.data;
};
```

Este patrón se repite en todos los servicios, manteniendo una estructura limpia y coherente.

11. Buenas prácticas y recomendaciones

El proyecto ha sido desarrollado siguiendo estándares profesionales que garantizan calidad, mantenibilidad y escalabilidad. A continuación se detallan las principales buenas prácticas aplicadas:

11.1 Organización del código

- Separación clara entre componentes, hooks, servicios y utilidades.
- Uso de nombres descriptivos y coherentes.
- Modularización de lógica compleja en hooks personalizados.
- Estructura de carpetas alineada con el flujo funcional del sistema.

11.2 Validaciones estrictas

- Uso de **Zod** para definir esquemas de validación tipados.
- Integración con **React Hook Form** para formularios reactivos y seguros.
- Mensajes de error específicos y retroalimentación visual inmediata.
- Bloqueo de acciones si los datos no cumplen con la ley.

11.3 Manejo de datos eficiente

- Uso de **React Query** para consumir, cachear y sincronizar datos con el backend.
- Evita llamadas innecesarias y mejora el rendimiento.
- Manejo automático de estados de carga, error y éxito.
- Refetch automático tras operaciones CRUD.

11.4 Estilos coherentes

- Uso de **Tailwind CSS** para estilos utilitarios y personalizados.
- Consistencia visual en botones, formularios, tablas y modales.
- Adaptación responsiva para distintos tamaños de pantalla.
- Paleta de colores institucional y tipografía legible.

11.5 Seguridad y control de acceso

- Autenticación mediante JWT con validación en frontend.
- Roles diferenciados que controlan el acceso a funcionalidades.
- Redirección automática en caso de acceso no autorizado.
- Almacenamiento seguro del token en localStorage.

11.6 Flujo de trabajo con Git

- Commits descriptivos y frecuentes.
- Uso de ramas para nuevas funcionalidades.
- Integración continua mediante pull requests.
- Documentación clara en el README.md..

12. Contribución y mantenimiento

El proyecto está diseñado para facilitar la colaboración entre desarrolladores, técnicos y personal académico. A continuación se detallan las pautas para contribuir al sistema y mantener su calidad a lo largo del tiempo.

12.1 Cómo colaborar

- **Clonar el repositorio** desde GitHub y crear una rama nueva para cada funcionalidad o corrección.
- **Realizar commits frecuentes** con mensajes descriptivos que expliquen claramente los cambios realizados.
- **Ejecutar pruebas manuales** antes de subir cambios, verificando que no se rompa ninguna funcionalidad existente.
- **Aplicar formato y linting** con los scripts npm run format y npm run lint antes de cada push.
- **Enviar pull requests** con una descripción clara del propósito, cambios realizados y pruebas realizadas.
- **Responder a revisiones** de forma colaborativa y abierta, manteniendo el enfoque en la mejora continua.

12.2 Flujo de trabajo recomendado

git checkout -b feature/nueva-funcionalidad

```
npm run lint
```

```
npm run format
```

```
git add .
```

```
git commit -m "Agrega validación de salario mínimo por sector"
```

```
git push origin feature/nueva-funcionalidad
```

Este flujo garantiza orden, trazabilidad y control de calidad en cada contribución.

12.3 Mantenimiento del sistema

- **Actualizar dependencias** periódicamente para mantener seguridad y compatibilidad.
- **Revisar cambios legales** en la normativa guatemalteca y ajustar validaciones si es necesario.
- **Documentar nuevas funcionalidades** en el manual técnico y en el README.md..
- **Realizar auditorías internas** para verificar cumplimiento legal y técnico.
- **Mantener coherencia visual** en nuevos componentes y vistas.

15. Cierre

Este manual técnico representa un esfuerzo por documentar con precisión y profesionalismo cada aspecto del frontend del sistema de nóminas. Su propósito es servir como guía de referencia para desarrolladores, evaluadores y colaboradores que deseen comprender, mantener o escalar el sistema.

La estructura modular, el enfoque legal, el uso de tecnologías modernas y la documentación detallada reflejan un compromiso con la calidad, la transparencia y la mejora continua.